

BhashaGuard Edge

On-Device Cultural Hallucination Guard for Indian-Language AI

Solution proposal | Snapdragon® AI Lab Build & Present Challenge 2026

One-line pitch: An offline review layer that checks Indian-language AI responses for script violations and evidence-backed cultural mismatches, then explains what is supported, contradicted or unresolved.

Problem and intended user

An AI response can use the correct script while confusing a local practice, place, person or cultural context. A teacher reviewing such a response needs to see the questionable claim and the evidence behind it. A valid regional variation must not be labelled false merely because it is missing from a model's knowledge.

BhashaGuard Edge focuses on teachers, students and local-language content reviewers. The first use case is reviewing an AI-written Telugu cultural learning note before it is shared. Tamil and Kannada are planned extensions after native-speaker validation.

Proposed solution

The application will accept a prompt and a pasted or locally generated answer, together with the requested language and locale. It will check script fidelity, extract factual claims, retrieve passages from a reviewed local knowledge pack, and identify support, contradictions, prompt/context mismatches and missing evidence. A human-readable report will show the original text spans, source passages and reasons for each finding.

The design keeps user text and inference on the PC after model and knowledge-pack installation. Where evidence is insufficient or conflicting, the system will abstain from a factual verdict. The user remains responsible for accepting or correcting the content.

Research evolution and differentiation

This proposal develops my existing research direction on a Cultural Hallucination Benchmark for Dravidian languages into an application. The proposed additions are a runtime checking pipeline, a versioned evidence pack, an interactive review workflow and a Snapdragon deployment and evaluation path.

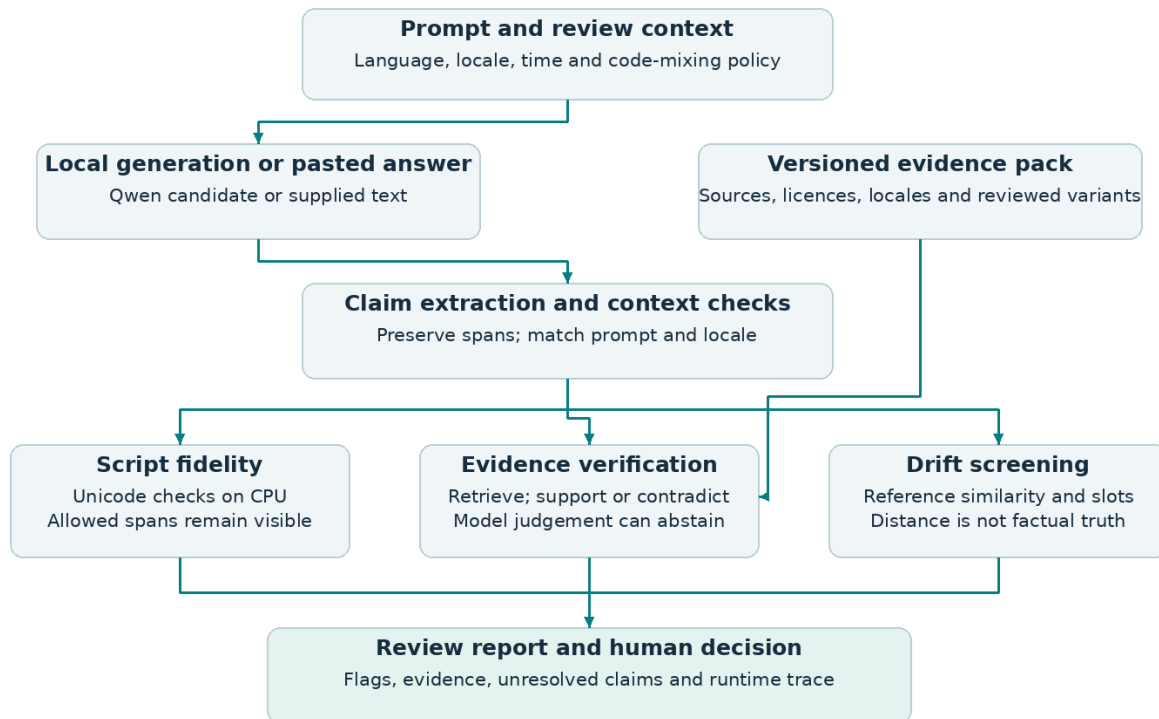
Its distinguishing focus is the combination of Dravidian-language error categories, locale-qualified evidence, legitimate regional variants and explicit uncertainty. Indian-language hallucination benchmarks already exist, including BHRAM-IL; the contribution here is the proposed application and evaluation design, not a claim to have invented multilingual hallucination detection. [8]

Submission status: This is a solution proposal. The architecture, deployment and evaluation below are planned work. No measured BhashaGuard accuracy, latency, memory usage or NPU speedup is claimed.

Technical architecture

BhashaGuard Edge architecture

PROPOSED • Local processing after setup • Acceleration subject to device validation



1. Capture context. Record the requested language, locale, time period and permitted code-mixing. Preserve the original answer and text offsets. A language will not be treated as a unique cultural or regional identity.

2. Check script. Use Unicode Script and Script_Extensions properties on a normalized working copy, with explicit handling of punctuation, combining marks and permitted quotations. Script compliance does not prove language identity or factual correctness. [7]

3. Extract and retrieve. Segment checkable claims and retrieve evidence using entity aliases and multilingual embeddings. Each passage will include a source URL, excerpt, locale, date, licence, reviewer and knowledge-pack version.

4. Verify claims. Use rules for scoped entity-relation facts and structured local-model judgements for open text. Require evidence IDs and exact cited spans; reject invalid citations and malformed output. Compare claims with the prompt's entities, locale and time.

5. Measure drift and uncertainty. Use reference-answer similarity as a screening signal, alongside evidence conflicts in cultural entities or relations. Missing evidence, disputed sources and ambiguous variants will produce an unresolved state.

6. Report for review. Show script issues, supported claims, contradictions, valid variants and unresolved claims separately. Template-based explanations will cite evidence. The reviewer can annotate findings and export a readable report without silently rewriting the answer.

Snapdragon implementation and model mapping

Target platform: Snapdragon-powered HP PCs running Windows ARM64, initially targeting the Snapdragon X Elite class. The planned implementation separates CPU text processing, storage and UI from neural stages eligible for Hexagon NPU execution. Windows ARM64 routes are documented in GenieX and ONNX Runtime QNN. [2, 5]

Pipeline stage	Model or method	Execution plan
Generation and open-text verification	Qwen3-4B-Instruct-2507; official Qualcomm AI Hub recipe. [1]	Matching precompiled W4A16 bundle through GenieX QAIRT, subject to exact-chipset availability and validation.
Smaller-model comparison	Qwen3-0.6B; official AI Hub recipe. [3]	Compare resource usage and judgement quality; do not assume the smaller model is adequate.
Retrieval and semantic drift	intfloat/multilingual-e5-small; model card lists Telugu, Tamil and Kannada. [4]	CPU baseline, followed by a proposed custom ONNX/QNN port. A ready-made AI Hub E5 asset is not assumed.
Script, metadata and reporting	Unicode properties, SQLite and deterministic application logic. [7]	CPU. These stages do not require NPU acceleration.

Compatibility boundary. The exact model bundle, SoC, runtime, driver and language quality must be validated before reporting NPU operation; a model listing alone does not establish compatibility with the target PC. GenieX is a developer preview; its QAIRT bundles are chipset-specific and NPU-only. [2, 9]

Optimization and verification plan

First establish a correct CPU pipeline. Preserve E5 tokenization, pooling, normalization and query/passage prefixes. For the proposed NPU port, export fixed input shapes and calibrate with representative development data, then compile and profile through AI Hub. Workbench supports ONNX/PyTorch inputs and SoC-specific QNN deployment artifacts. [4, 6]

Record the HP device and SoC, model revision/hash, precision, input limits, Windows build and runtime versions. Validate output quality after conversion. Use backend/operator traces to establish actual HTP execution and disclose mixed CPU/NPU operation; loading the QNN provider alone does not prove NPU use. [5]

Why on-device AI

After initial setup, the intended workflow requires no external inference service. Local processing can keep classroom drafts and reference material on the laptop and avoid an inference-network dependency. The prototype will test this with networking disabled. Model downloads and optional AI Hub development jobs are separate from offline application use; private user drafts will not be used as remote profiling inputs.

Evaluation methodology and measurable outcomes

Evaluation will measure checking quality and device performance separately. The following definitions are proposed metrics for this project; they do not certify cultural truth.

Metric	Definition and interpretation
Script fidelity	$S = \text{target-script alphabetic code points} / \text{assessed alphabetic code points}$. Apply a fixed exclusion policy for approved spans. Report counts; return N/A for an empty denominator.
Evidence consistency and coverage	For s supported, c contradicted and u unresolved claims, $N = s + c + u$. Consistency = $s / (s + c)$; coverage = $(s + c) / N$. Show both and return N/A for zero denominators.
Semantic and cultural drift	Semantic distance = $(1 - \text{maximum cosine similarity to reviewed acceptable reference answers}) / 2$. Use only as a screening feature. Separately count evidence-conflicting entity/relation/locale/time slots.
Risk output	Begin with categorical flags and abstentions. Fit a calibrated probability of a material error only after sufficient labelled data. Report calibration on held-out cases; do not present arbitrary weights as probabilities.
Device performance	Median and p95 guard latency; time to first token; decode tokens/second; peak process memory; model/index disk size; cold-start time; and load failures.

Quality evaluation

Start with a manually reviewed Telugu smoke set. For a later study, plan 300 prompt-answer cases per language, covering supported content, contradictions, script errors, missing evidence, context mismatch and valid regional variants. This is a planned dataset size, not a completed benchmark. Include code-mixing, transliteration, negation and multi-claim answers.

Use two native-language annotators with adjudication. Split by topic/entity and source family to reduce leakage. Freeze the test set and evidence pack; keep test labels out of prompts and rules. Compare script-only checks, similarity-only retrieval, an ungrounded local-model judge and the complete guard. Report macro-F1, per-language precision/recall, false alarms on valid variants, claim-extraction recall, retrieval Recall@k, citation validity and abstention versus error rate.

CPU and NPU comparison

Use the same HP device, fixed inputs, batch size one and recorded power settings. Separate cold start from warm runs; use five warm-ups and at least 30 timed runs per input-length group. Compare equivalent model revisions, shapes, precision and quality. Speedup = median CPU latency / median NPU latency. When artifacts differ, label the result a deployment comparison. QNN-only binaries require a separate compatible CPU artifact. [5, 6]

Deployment, demonstration and scope

The prototype will use a lightweight interface and a local service, with SQLite metadata and a compact embedding index. Fonts, UI assets, models and the reviewed knowledge pack will be installed locally. Exports will include findings, sources, pack/model versions and runtime timings. Private drafts will not be retained by default.

Planned demonstration

A teacher selects Telugu and a locale, then analyses a labelled test paragraph. The demo shows a script violation, one evidence-backed contradiction and one claim that cannot be verified. The reviewer opens the cited passage, inspects a legitimate regional-variant case and exports a report. English translations will accompany Indian-language examples. The final screen will display the actual backend and measured timings; fixtures and CPU runs will be labelled accurately.

Limitations and next milestones

A limited knowledge pack can miss valid traditions. Retrieval, translation and claim-extraction errors may affect the verdict, and a local verifier can itself hallucinate. Semantic similarity is not entailment. The application will abstain on conflicting or insufficient evidence and retain human review. Uploaded source text will be treated as data, not instructions; fabricated evidence IDs will be rejected.

The first milestone is a Telugu text-only review pipeline with reproducible CPU evaluation. The next is validated Snapdragon inference and a measured comparison. Tamil and Kannada, stronger language identification, dialect/transliteration handling, voice input and evidence-pack updates will follow language-specific validation. Third-party models and evidence will retain their licences and attribution.

Alignment with the challenge

The proposal addresses technical implementation through a modular checking and deployment plan; use case and innovation through cultural claim review; deployment and accessibility through offline HP-PC operation; and presentation/documentation through a traceable report and reproducible evaluation design. These correspond to the published judging categories. [10]

References

1. [Qualcomm Qwen3-4B-Instruct-2507 recipe](#)
2. [GenieX overview and runtime documentation](#)
3. [Qualcomm Qwen3-0.6B recipe](#)
4. [Multilingual E5 small model card](#)
5. [ONNX Runtime QNN execution provider](#)
6. [Qualcomm AI Hub Workbench compilation documentation](#)
7. [Unicode Standard Annex 24](#)
8. [BHRAM-IL benchmark, ACL Anthology](#)
9. [AI Hub Qwen3-4B-Instruct-2507 Compute listing](#)
10. [Qualcomm challenge listing on Unstop](#)